

AI-Architect Architecture Report

Project: Gemini Greenfield

Run ID: 20260828T123928Z-9445e65e

Status: Accepted — 2026-08-28 13:00 UTC

Final Review Verdict: PASS

Models used: gemini-3.1-flash-lite

Executive Summary

The Gemini Greenfield platform is designed to handle high volumes of traffic during peak seasons, ensuring a responsive experience for our customers. By using a modular, service-based architecture, we can independently scale specific areas of the business (like search or checkout) based on demand, enabling cost-efficient resource utilization. This approach accelerates time-to-market for new features, allowing us to pivot or expand capabilities rapidly to meet changing customer needs, with built-in cost transparency and optimization controls.

Requirements & Constraints

Business Goal

Build a new e-commerce platform that can support future growth and high peak traffic.

Problem Statement

The current platform lacks the scalability to handle anticipated high-traffic spikes and is too rigid to support the rapid feature development required for our future growth.

Users / Stakeholders

- Online customers, customer support, warehouse and inventory staff, finance and payment operations, product management, software engineers, and operations staff.

Functional Requirements

- Browse and search products
- Manage customer accounts
- Manage shopping carts
- Place and process orders
- Handle payments
- Manage inventory
- View order status and history

Non-Functional Requirements

- Support up to 50,000 concurrent users during peak campaigns
- Maintain high availability during peak periods
- Keep customer-facing interactions responsive
- Allow the system to scale as demand grows

Cloud Provider

AWS

Budget

No fixed monetary budget has been defined. Cost efficiency should be considered.

Recommended Architecture

Pattern

Microservices with Event-Driven Communication

Rationale

To meet significant peak traffic requirements and support rapid feature iteration, a microservices pattern provides the necessary fault isolation and independent scaling capabilities. Event-driven communication handles backend processes asynchronously, improving responsiveness and decoupling service dependencies. The platform adopts an explicit FinOps and cost-efficiency strategy, utilizing managed AWS services, automated rightsizing, and rigorous resource tagging to align with the unconstrained budget while ensuring cost optimization.

Components

ID	Name	Type	Purpose
COMP-API-GATEWAY	API Gateway	API	Single entry point for client requests, routing to internal microservices.
COMP-PRODUCT-SERVICE	Product Service	service	Manage the product catalog and search functionality.
COMP-IDENTITY-SERVICE	Identity Service	service	Manage customer accounts and authentication.
COMP-CART-SERVICE	Cart Service	service	Manage temporary shopping cart sessions.
COMP-ORDER-SERVICE	Order Service	service	Orchestrate order lifecycle.
COMP-PAYMENT-SERVICE	Payment Service	service	Handle payment transactions.
COMP-INVENTORY-SERVICE	Inventory Service	service	Manage stock levels.
COMP-EVENT-BUS	Event Bus	queue	Facilitate asynchronous communication.

COMP-API-GATEWAY: API Gateway

The API Gateway manages client traffic, authentication, and routing. It ensures that internal service structures remain abstracted from the end user.

Technology

AWS API Gateway

Traces to features

FEAT-001, FEAT-002, FEAT-003, FEAT-004, FEAT-005, FEAT-006

Justified by

ADR-004, ADR-005

COMP-PRODUCT-SERVICE: Product Service

Responsible for storing and retrieving product information. Provides search and browsing

capabilities.

Technology

Node.js, Document Store (e.g., MongoDB/DynamoDB)

Traces to features

FEAT-001

Justified by

ADR-001, ADR-003, ADR-005

COMP-IDENTITY-SERVICE: Identity Service

Handles user registration, authentication, and profile updates.

Technology

Java, Relational Database

Traces to features

FEAT-002

Justified by

ADR-001, ADR-003, ADR-005

COMP-CART-SERVICE: Cart Service

Maintains the state of customer shopping carts during a session.

Technology

Redis, Node.js

Traces to features

FEAT-003

Justified by

ADR-001, ADR-003, ADR-005

COMP-ORDER-SERVICE: Order Service

Processes new orders, tracks order status, and manages order history.

Technology

Java, Relational Database

Traces to features

FEAT-004, FEAT-005

Justified by

ADR-001, ADR-002, ADR-003, ADR-005

COMP-PAYMENT-SERVICE: Payment Service

Integrates with payment providers to process and authorize payments.

Technology

Java

Traces to features

FEAT-004

Justified by

ADR-001, ADR-002, ADR-003, ADR-005

COMP-INVENTORY-SERVICE: Inventory Service

Tracks inventory levels and reserves stock during the order process.

Technology

Java

Traces to features

FEAT-004

Justified by

ADR-001, ADR-002, ADR-003, ADR-005

COMP-EVENT-BUS: Event Bus

Broker for inter-service events ensuring decoupling between Order, Payment, and Inventory services.

Technology

Amazon SNS/SQS or Kafka

Traces to features

FEAT-004, FEAT-006

Justified by

ADR-002, ADR-005

Data Flows

- Customer → API Gateway
- API Gateway → Product Service
- API Gateway → Identity Service
- API Gateway → Cart Service
- API Gateway → Order Service
- Order Service → Event Bus
- Event Bus → Payment Service
- Event Bus → Inventory Service
- Payment Service → Event Bus
- Inventory Service → Event Bus

Architecture Decision Records

ADR-001: ADR-001: Adopt Microservices Architecture

Status: accepted

Context

The platform needs to support rapid feature development and handle scaling requirements effectively.

Decision

Use a microservices-based architecture to decouple business domains.

Rationale

Microservices allow independent scaling and fault isolation, preventing bottlenecks in the monolithic structures that limit current growth.

Alternatives Considered

- Monolithic architecture

Positive Consequences

- Independent deployment cycles
- Fine-grained scaling
- Fault isolation

Negative Consequences / Trade-offs

- Increased operational complexity
- Need for sophisticated inter-service communication

Related Features: FEAT-001, FEAT-002, FEAT-003, FEAT-004, FEAT-005, FEAT-006

Related Components: Product Service, Identity Service, Cart Service, Order Service, Payment Service, Inventory Service

Related Decision Topics: TOPIC-1

Evidence: KB-E001, KB-E003, KB-E004

ADR-002: ADR-002: Hybrid Communication Pattern

Status: accepted

Context

Requirement for high-performance and reliable processing during peak traffic.

Decision

Use synchronous REST/GraphQL for user-facing interactions and asynchronous message-based communication for background processes.

Rationale

Synchronous communication provides immediate feedback for UI; asynchronous events provide decoupling and fault tolerance for backend processes.

Alternatives Considered

- Pure synchronous communication
- Pure asynchronous communication

Positive Consequences

- Improved system availability
- Better throughput during load spikes

Negative Consequences / Trade-offs

- Eventual consistency challenges
- Complexity in debugging event flows

Related Features: FEAT-004, FEAT-006

Related Components: Order Service, Event Bus, Payment Service, Inventory Service

Related Decision Topics: TOPIC-3

Evidence: KB-E001, KB-E002

ADR-003: ADR-003: Distributed Data Persistence

Status: accepted

Context

Microservices need autonomy and data isolation.

Decision

Each service manages its own database schema, strictly separating data ownership.

Rationale

Database per service prevents shared data bottlenecks and allows each service to choose the storage technology suited for its specific data model.

Alternatives Considered

- Shared database

Positive Consequences

- Data isolation
- Independent scalability of storage

Negative Consequences / Trade-offs

- Cross-service data consistency requires orchestration or choreography

Related Features: FEAT-001, FEAT-002, FEAT-003, FEAT-004

Related Components: Product Service, Identity Service, Cart Service, Order Service, Payment Service, Inventory Service

Related Decision Topics: TOPIC-2

Evidence: KB-E001, KB-E004, KB-E005

ADR-004: ADR-004: Horizontal Auto-Scaling Strategy

Status: accepted

Context

Requirement to support high, fluctuating peak traffic volumes while maintaining cost efficiency.

Decision

Implement horizontal auto-scaling for all services on AWS infrastructure.

Rationale

Horizontal scaling allows the platform to adapt resources to real-time traffic, ensuring system responsiveness during peaks while minimizing costs during low-demand periods through elastic resource management.

Alternatives Considered

- Vertical scaling

Positive Consequences

- Cost-effective resource usage
- High availability during traffic bursts

Negative Consequences / Trade-offs

- Need for automated health checks and traffic management

Related Features: FEAT-006

Related Components: API Gateway, Product Service, Identity Service, Order Service

Related Decision Topics: TOPIC-4

Evidence: KB-E001, KB-E004, KB-E005

ADR-005: ADR-005: Cost-Efficiency and FinOps Strategy

Status: accepted

Context

Budget is undefined but cost-efficiency is a hard requirement. The platform must manage cloud costs dynamically while scaling.

Decision

Implement a FinOps framework including AWS resource tagging for cost attribution, automated rightsizing policies, and AWS Budgets alerts.

Rationale

Active cost management and rightsizing are essential to meet the mandate for cost efficiency while maintaining performance under scale.

Alternatives Considered

- Manual cost reviews
- Fixed over-provisioning

Positive Consequences

- Reduced cloud waste
- Improved cost transparency

Negative Consequences / Trade-offs

- Requires engineering investment in automation
- Complexity in tagging policies

Related Features: FEAT-006

Related Components: API Gateway, Product Service, Identity Service, Cart Service, Order Service, Payment Service, Inventory Service, Event Bus

Related Decision Topics: TOPIC-4

Evidence: KB-E001

Evidence / Literature

Curated Knowledge Base Evidence

ID	Source	Page	Excerpt
KB-E001	Rag Database/box2_domain/ecommerce_migration_event_driven_bulus.md	0	Microservices architecture has emerged as a compelling alternative, offering the promise of enhanced scalability, improved fault tolerance, and greater development agility [1]. By decomposing monolithic applications into smaller, independently deployable services, organizations can achieve better resource utilization,...
KB-E002	Rag Database/box2_domain/ecommerce_migration_event_driven_bulus.md	0	**b. High Load Scenarios:** Asynchronous patterns (message queues) demonstrate superior throughput and availability [4]. **c. Availability:** Event-driven architectures provide better fault tolerance and system availability under stress [4]. These findings suggest that e-commerce systems should employ hybrid approac...

ID	Source	Page	Excerpt
KB-E003	Rag Database/box2_domain/ecommerce_microservices_challenges_ibrahim_luong.md	0	## 2.2 Monolithic and Microservices Architectures Comparison *(printed p. 478)* In the e-commerce landscape, choosing between microservices and traditional monolithic architectures requires careful consideration of various factors. Monolithic architectures, characterized by their single, tightly coupled codebase. How...
KB-E004	Rag Database/box2_domain/ecommerce_migration_event_driven_bulus.md	0	## Introduction — relevante Absätze zur E-Commerce-Migration *(printed p. 2)* The rapid evolution of e-commerce platforms has necessitated architectural paradigm shifts to meet increasing demands for scalability, reliability, and agility [2]. Traditional monolithic architectures, while providing simplicity in develop...
KB-E005	Rag Database/box2_domain/ecommerce_microservices_challenges_ibrahim_luong.md	0	ensuring the security of microservices and their interactions is crucial in e-commerce, where sensitive customer data is often processed. Abgaz et al. [6] have also stated the importance of strong security and identity management inside a microservices ecosystem. Scalability is another challenge, as e-commerce platfor...

Validation & Reviewer Findings

Overall Verdict: PASS

Refinement rounds: 2

No open findings were recorded on the accepted review.

Traceability

Features -> Decisions / Components

Feature	Name	ADRs	Components
FEAT-001	Product Catalog Exploration	ADR-001, ADR-003	COMP-API-GATEWAY, COMP-PRODUCT-SERVICE
FEAT-002	Customer Account Management	ADR-001, ADR-003	COMP-API-GATEWAY, COMP-IDENTITY-SERVICE
FEAT-003	Shopping Cart Management	ADR-001, ADR-003	COMP-API-GATEWAY, COMP-CART-SERVICE
FEAT-004	Order Fulfillment and Payment Processing	ADR-001, ADR-002, ADR-003	COMP-API-GATEWAY, COMP-ORDER-SERVICE, COMP-PAYMENT-SERVICE, COMP-INVENTORY-SERVICE, COMP-EVENT-BUS
FEAT-005	Order Tracking	ADR-001	COMP-API-GATEWAY, COMP-ORDER-SERVICE
FEAT-006	System Performance and Scalability	ADR-001, ADR-002, ADR-004, ADR-005	COMP-API-GATEWAY, COMP-EVENT-BUS

Decision Topics -> ADRs

Topic	Question	ADRs
TOPIC-1	service decomposition and boundaries	ADR-001
TOPIC-2	data ownership and persistence strategy	ADR-003
TOPIC-3	integration style: synchronous vs asynchronous communication	ADR-002
TOPIC-4	scaling and availability strategy	ADR-004, ADR-005

ADRs -> Evidence

ADR	Evidence
ADR-001	KB-E001, KB-E003, KB-E004
ADR-002	KB-E001, KB-E002
ADR-003	KB-E001, KB-E004, KB-E005
ADR-004	KB-E001, KB-E004, KB-E005
ADR-005	KB-E001

Limitations / Open Items

No unresolved items are recorded on this run.